Project Planning & Management

Smart Budget Travel & Fare Optimizer

Track: DEPI Data Engineering | **Date:** October 2026
Team Members: Yassin Alaa El Din (Lead), Jana Khaled, Peter Fadel, Mohamed Khaled
Document Status: Official Team Governance & Execution Plan

1. PROJECT OVERVIEW & OBJECTIVES

The Smart Budget Travel & Fare Optimizer project constructs an end-to-end data platform combining multi-source web scrapers, time-series forecasting, automated pipeline orchestration, and an interactive travel recommender dashboard. The goal is to maximize booking savings for travelers through data transparency and fare prediction.

2. MEASURABLE SUCCESS CRITERIA

- **Automated Ingestion:** Airflow DAGs execute end-to-end scraping and ETL without manual intervention.
- **Data Warehouse Integrity:** Dimensional star schema loaded into PostgreSQL passes all primary/foreign key tests.
- **Forecasting Accuracy:** Price prediction models achieve target error metrics (RMSE/MAE) on historical travel datasets.
- **Interactive Dashboard:** Streamlit UI dynamically filters package bundles and renders fare trend charts under 3 seconds.

3. TASK ALLOCATION MATRIX (RACI)

Team Member	Primary Role	Core Domain	Secondary Ownership
Yassin Alaa El Din	Team Lead	Airflow Pipelines & Database Architecture	System Integration
Jana Khaled	Data Engineer	Web Scraping & API Ingestion	ETL Data Cleaning
Peter Fadel	ML Engineer	Time-Series Forecasting Models	Data Profiling & EDA
Mohamed Khaled	BI / Full-Stack	Streamlit Dashboard & Package Recommender	Data Quality UI

4. SCHEDULE & WEEKLY MILESTONES

- **Weeks 1-2:** Project setup, GitHub repository, Kaggle benchmark EDA, and baseline ML training.
- **Weeks 3-4:** Custom scraper development (BeautifulSoup/Selenium) and API ingestion setup.
- **Weeks 5-6:** PostgreSQL dimensional warehouse design, ETL script creation, and Airflow orchestration.
- **Weeks 7-8:** Time-series prediction model refinement, Streamlit dashboard development, and DQ integration.
- **Week 9:** System integration testing, final code freeze, documentation, and final DEPI delivery.

5. TECHNICAL STACK & INFRASTRUCTURE

- **Programming & Core Libraries:** Python 3.10+, Pandas, NumPy, Scikit-Learn, Prophet.
- **Data Extraction:** BeautifulSoup, Selenium, Scrapy, Requests.
- **Data Storage:** MongoDB (Staging Unstructured Data), PostgreSQL (Structured Dimensional Warehouse).
- **Orchestration & Packaging:** Apache Airflow, Docker, Docker Compose.
- **Visualization & User Interface:** Streamlit, Plotly.

6. RISK MANAGEMENT & MITIGATION STRATEGIES

- **Risk 1 - Web Scraper IP Blocking (High Impact):** Mitigate by using dynamic User-Agents, request rate limiting, and leaning heavily on official REST APIs.
- **Risk 2 - Dynamic Fare Volatility (Medium Impact):** Mitigate by implementing timestamped snapshot tables and rolling price baselines in PostgreSQL.
- **Risk 3 - Integration Delays (High Impact):** Mitigate through weekly Agile sprints and maintaining a strict Git feature-branch workflow.